

Universidad Tecnológica Nacional.
Tecnicatura Universitaria en Programación. 1Pro1.
7 de junio del 2026.
Programación II.
Docente Roco.
Alumnos Danilo Serrano,
Jesús Ramírez y Lautaro Fernandez.

Trabajo Práctico Integrador:
Juego de Naipes Españoles.
Primera parte.

Capitán del equipo:

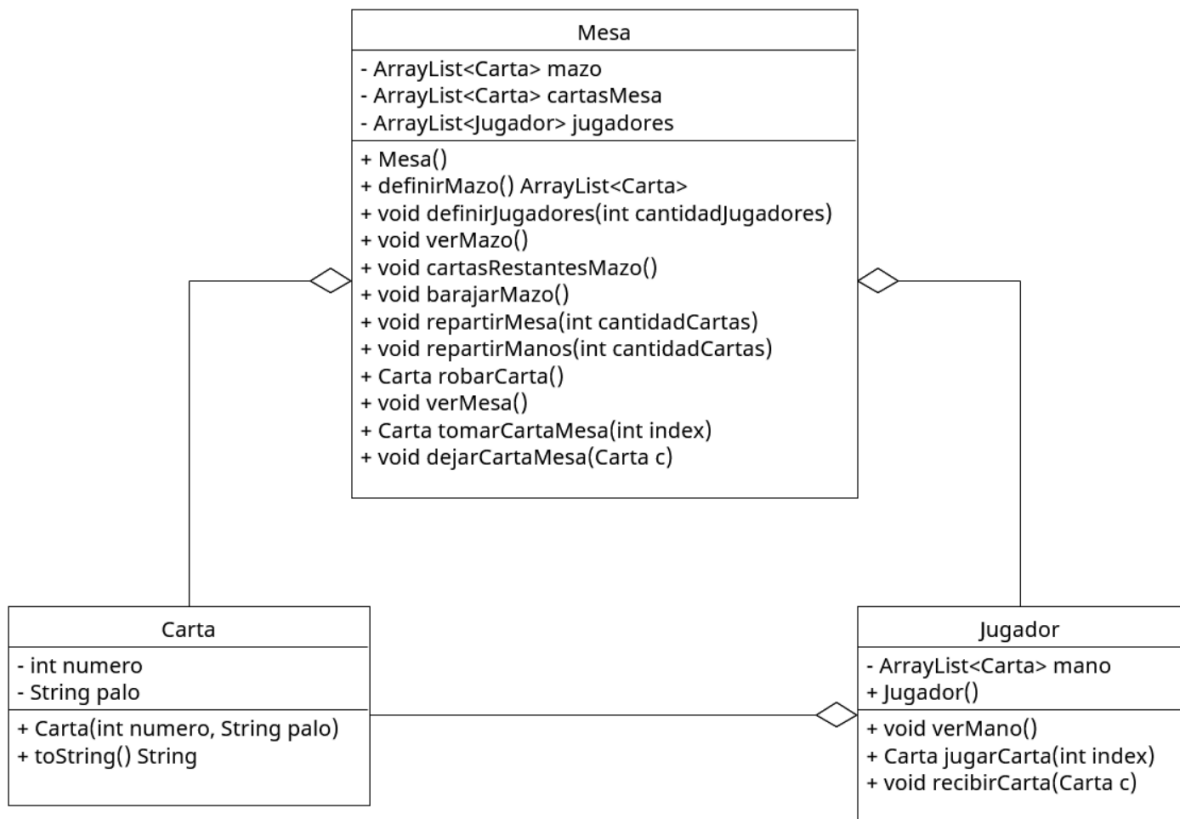
Danilo Serrano.

Análisis del problema.

El objetivo de este proyecto es diseñar e implementar una aplicación en Java que simule un juego con la baraja española tradicional de 40 cartas, debe tener los cuatro palos tradicionales (Oro, Bastos, Copas y Espadas) y los números del 1 al 12, excluyendo los ochos y los nueves. Actualmente, la solución se modela a través de cuatro clases donde la lógica se encuentra fuertemente centralizada en la clase Mesa, la cual actúa como motor principal al fabricar el mazo y registrar a los integrantes. Por su parte, la clase Carta funciona de manera inmutable almacenando el valor y palo de cada naipes, la clase Jugador gestiona las manos de los participantes, y la clase Main ejecuta el bucle de turnos mediante un menú interactivo por consola.

Para administrar el constante movimiento de los naipes, se justificó el uso de la estructura dinámica ArrayList sobre los arrays, permitiendo que el volumen de cartas en el mazo, la mesa y las manos de los jugadores cambie en tiempo real sin generar índices vacíos en la memoria. El diseño optimiza el rendimiento del código al utilizar funciones nativas como `Collections.shuffle()` para el mezclado y el método `removeLast()` para eliminar el último elemento sin que Java deba ajustar constantemente la dimensión de los ArrayLists que contienen las cartas. La Mesa es donde se manejan todas las clases para conseguir un juego de mesa de prueba.

Diagrama UML.



Justificación de las decisiones de diseño.

Como en Java los objetos lo son todo, lo primero que se hizo fue identificar las entidades esenciales que participan en una partida de la casita robada, lo cual nos llevó a crear las clases Carta, Jugador y Mesa. Luego se crearon sus atributos lógicos. De esta forma, cada Jugador tiene un ArrayList de objetos Carta que es su atributo mano, y Mesa tiene un ArrayList de jugadores, de cartas y de cartas en la mesa, que son sus atributos jugadores, su mazo y sus cartasMesa, respectivamente. La clase Main, por otro lado, implementa el código del menú y la lógica de cambio de turnos (el juego está programado para que se puedan tener tantos jugadores como se desee).

La casita robada no está implementada todavía, puesto que faltan las cruciales casitas y las reglas del propio juego; todo lo que se puede hacer actualmente son acciones propias de un juego de cartas genérico que no representan apropiadamente las reglas del juego que se quiere replicar, como lo sería robar una carta de la mesa (en la casita robada solo se obtienen cartas cuando ambos jugadores se quedan sin cartas en la mano).

El proyecto se encuentra en una etapa primaria donde la clase Mesa centraliza la lógica operativa. Al principio se descartó la creación de una clase Mazo porque argüimos que un mazo era sólo un array de cartas, y que sus métodos no serían los suficientes como para ameritar su propia clase. En su lugar lo suplimos con mazo, un atributo de Mesa, que se ocupa de manejar el mazo, justo como maneja casi todo lo demás. Al terminar nos dimos cuenta de lo que evidencia el diagrama UML: la clase Mesa lo hace todo. Creamos una superclase cuyas líneas de código y métodos con considerablemente más que las demás. Si bien el trabajo que tenemos entre manos es simple y no necesita obligatoriamente estructuras complejas, esta desproporción de datos será lo primero que habrá que arreglar para la próxima versión de este proyecto.

Análisis sobre posibles patrones de diseño aplicables.

Tras evaluar los patrones de diseño para este proyecto, determinamos que la arquitectura del sistema no justifica incorporar estructuras complejas. El criterio principal fue evitar que un patrón de diseño, en este proyecto, empeore el desarrollo del software al hacerlo más complejo. Al tratarse de una simulación a pequeña escala con un mazo fijo de 40 naipes inmutables, introducir patrones tradicionales aumentaría la complejidad del código y del diagrama UML sin aportar un valor real ni optimizar la resolución del problema.

Por este motivo se descartó Factory Method, ya que todas las cartas pertenecen a un único tipo básico (Carta) sin variantes complejas que requieran una fábrica, e igualmente se rechazó Singleton porque, aunque se use un solo mazo, este patrón acoplaría el código de forma global y bloquearía la flexibilidad futura de simular partidas con múltiples mazos o mesas en paralelo. Para el alcance actual, las herramientas puras de la programación orientada a objetos bastan para garantizar una solución eficiente y prolija.

Capturas del programa en ejecución.

```
---Turno del Jugador 1.  
1. Ver mesa  
2. Ver mano  
3. Robar de la mesa  
4. Robar del mazo  
5. Dejar en la mesa  
6. Ver mazo  
7. Cerrar programa  
Op:
```

```
1. Ver mesa  
2. Ver mano  
3. Robar de la mesa  
4. Robar del mazo  
5. Dejar en la mesa  
6. Ver mazo  
7. Cerrar programa  
Op: 6  
---Cartas del Mazo.  
7 de Espadas  
11 de Oro  
6 de Bastos  
2 de Bastos  
10 de Copas  
2 de Oro  
10 de Espadas
```

```
---Turno del Jugador 1.  
1. Ver mesa  
2. Ver mano  
3. Robar de la mesa  
4. Robar del mazo  
5. Dejar en la mesa  
6. Ver mazo  
7. Cerrar programa  
Op: 1
```

```
---Cartas en la Mesa.  
0. 5 de Oro  
1. 1 de Copas  
2. 4 de Copas  
3. 2 de Copas
```

```
1. Ver mesa  
2. Ver mano  
3. Robar de la mesa  
4. Robar del mazo  
5. Dejar en la mesa  
6. Ver mazo  
7. Cerrar programa  
Op: 2  
---Mano.  
0. 3 de Copas  
1. 6 de Copas  
2. 5 de Copas
```

```
1. Ver mesa
2. Ver mano
3. Robar de la mesa
4. Robar del mazo
5. Dejar en la mesa
6. Ver mazo
7. Cerrar programa
Op: 4
Robaste: 7 de Bastos
```

```
---Turno del Jugador 2.
```

```
1. Ver mesa
2. Ver mano
3. Robar de la mesa
4. Robar del mazo
5. Dejar en la mesa
6. Ver mazo
7. Cerrar programa
```

```
Op: 5
```

```
---Mano.
```

```
0. 2 de Espadas
1. 11 de Espadas
2. 6 de Oro
```

```
Índice de la carta a dejar: 2
```

```
Dejaste: 6 de Oro
```

```
---Cartas en la Mesa.
```

```
0. 5 de Oro
1. 1 de Copas
2. 4 de Copas
3. 2 de Copas
4. 6 de Oro
```

Universidad Tecnológica Nacional.
Tecnicatura Universitaria en Programación. 1Pro1.
21 de junio del 2026.
Programación II.
Docente Roco.
Alumnos Danilo Serrano, Jesús Ramírez
Lautaro Fernandez y **Daniela Diaz**.

Trabajo práctico integrador:
Juego de naipes españoles.
Segunda parte.

Capitán del equipo:

Danilo Serrano.

Juego elegido.

La casita robada.

Cabe acotar también que a nuestro equipo se sumó Daniela Díaz.

Reglas del Juego.

La última vez no había una implementación del juego, pero ahora que el proyecto ha sido finalizado y sí se ha implementado la casita robada, pensamos necesario explicar las reglas de nuestra implementación (que se ciñe bastante a los reglamentos originales de la casita robada).

El juego se desarrolla con una baraja española de 48 cartas (ni los ochos ni los nueves fueron excluidos esta vez). En nuestra implementación, la sota, el caballero y el rey fueron reemplazados (en nombre) por 10, 11 y 12, respectivamente. Se juega de la siguiente forma:

- Se colocan cuatro cartas boca arriba en la mesa y se reparten 3 cartas a cada jugador.
- En su turno, cada jugador revisa si el número de alguna de las cartas en la mesa coincide con alguna que tengan en mano.
- Si tienen cartas coincidentes, pueden emparejar la carta de la mano con la de la mesa y mandar ambas a sus casitas (unas pilas de cartas boca arriba que empiezan vacía. Cada jugador posee una).
- Si no tienen cartas coincidentes o no quieren añadir nada a la casita, los jugadores pueden descartar una carta de sus manos para añadirla a la mesa.
- Si tienen una carta en mano que coincide con el número de la carta sobre la casita del rival, pueden robar la casita del rival para añadir todas sus cartas a sus casitas propias.
- Los jugadores solo reponen sus cartas cuando ambos se hayan quedado sin cartas en la mano. Se reparten siempre 3 cartas a cada uno.
- El juego termina cuando se agotan tanto las cartas del mazo como las de las manos de los jugadores. Gana quien tenga la mayor cantidad de cartas en su casita.

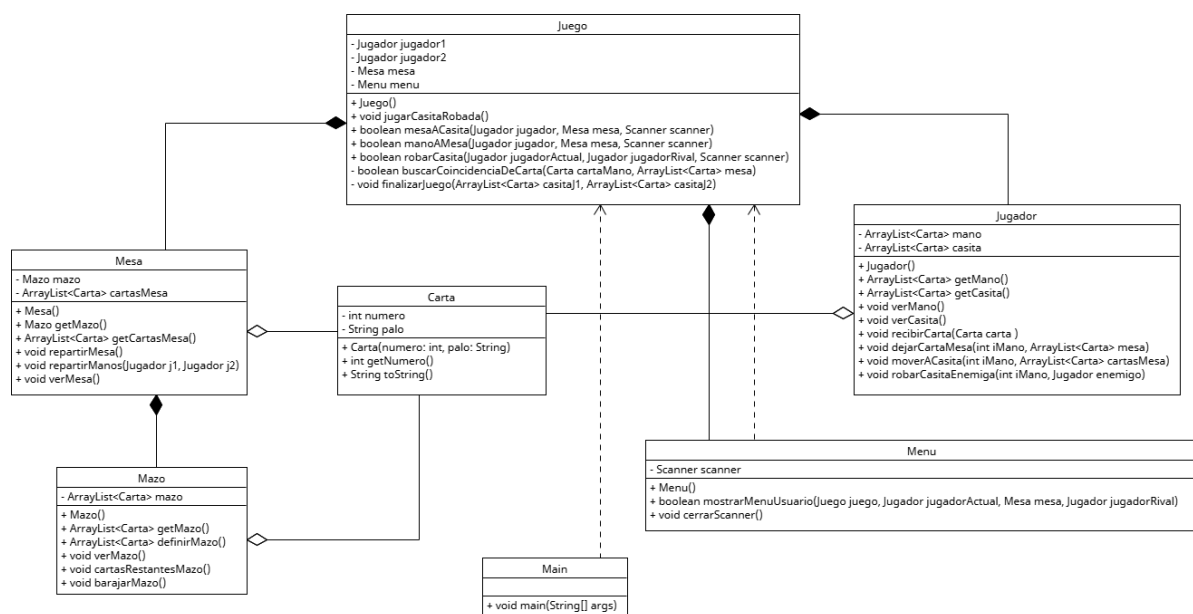
Análisis del problema.

El desarrollo de la casita robada mediante programación orientada a objetos requirió modelar un sistema de transacciones dinámicas de información entre arrays de objetos Carta. El desafío principal radicó en la desmantelación y redistribución de las funciones de la anteriormente omnipresente clase Mesa. Esto nos forzó a crear dos clases adicionales. La clase Main también sufrió un cambio importante; reparamos en que su única función debería ser la ejecución del código ya existente en otras clases, por lo que movimos todo lo referente al menú a su propia clase.

En definitiva, la parte más complicada fue idear formas de conseguir acceder a los métodos y atributos de varias clases desde otra clase ahora que Mesa no contiene un acceso rápido y sencillo a prácticamente todos los métodos.

A diferencia de otros juegos de naipes donde las cartas ganadas van a un pozo estático y cerrado, la mecánica fundamental de este juego exige que la casita sea una estructura indexada abierta y consultable en su capa superior por objetos externos (el jugador rival). Esto implicó diseñar métodos de transferencia masiva donde un objeto contenedor pierde la propiedad total de sus elementos en favor de otro en un solo ciclo de ejecución, requiriendo un control estricto de las referencias para evitar la duplicación de objetos o pérdidas de consistencia en el estado del mazo general.

Diagrama UML actualizado.



Este diagrama también será adjuntado a la carpeta en que vino este documento.

Justificación de las decisiones de diseño.

Se decidió descentralizar la lógica operativa que sobrecargaba a la entidad Mesa en la primera fase del proyecto.

Se separaron todos los métodos de mazo a su propia clase dedicada llamada Mazo.

Se introdujo la clase Juego, la cual actúa como el cerebro del sistema que coordina los eventos del juego (repartir, validar robos, coordinar turnos, coordinar el mazo, etc) llamando los métodos de otras clases o usando los suyos propios cuando conciernen directamente a la lógica del juego.

Asimismo, se aplicó una estricta separación de responsabilidades delegando la capa de presentación a la clase Menu. Esta última opera únicamente mediante el paso de parámetros con el fin de maximizar la cohesión de las clases principales (Jugador, Mesa, Mazo), las cuales se limitan a ser contenedores de datos con métodos que alteran su propio estado, respetando el principio de diseño de que cada clase debe hacer una sola cosa bien.

A parte de esto y de la adición de métodos para la implementación de las reglas del juego (como lo son `mesaACasita()`, `manoAMesa()` y `robarCasita()`), el cómo opera el proyecto sigue siendo bastante parecido: todavía se emplean `ArrayLists` para la administración de las manos, los mazos y las cartas en la mesa (aunque se añaden más para las casitas). Un cambio importante es que ahora el juego está limitado a dos jugadores, y no los técnicamente ilimitados que había en la versión pasada. Por lo mismo, ambos son objetos de la clase Juego.

Como comentario adicional, queremos aclarar que existe una forma de ver el mazo sólo por conveniencia a la hora de revisar que el código funciona apropiadamente.

Patrones de diseño.

Para el diseño de esta segunda etapa, se analizaron alternativas arquitectónicas con el fin de estructurar el flujo de control. Se analizó la inclusión del patrón State para manejar la alternancia de turnos y las fases de reparto, o el patrón Command para encapsular cada jugada del usuario. Sin embargo, dada la escala del dominio y el requerimiento de no sobrecomplicar la solución, se optó por no aplicar un patrón de diseño.

Capturas de ejecución.

```
-----Turno del JUGADOR 1
```

```
1. Ver mesa y mano.  
2. Mover a casita.  
3. Dejar en la mesa.  
4. Robar casita.  
5. Ver casitas.  
6. Ver mazo.  
7. Cerrar programa.  
Opción:
```

```
Opción: 1
```

```
--- Cartas en la Mesa.
```

```
0. 10 de Copas  
1. 3 de Copas  
2. 11 de Copas  
3. 2 de Bastos
```

```
--- Mano.
```

```
0. 6 de Copas  
1. 8 de Bastos  
2. 7 de Espadas
```

```
Opción: 5
```

```
[MIA] --- Casita.  
La casita está vacía.  
[RIVAL] --- Casita.  
Carta superior: 8 de Espadas.  
Tamaño: 4 cartas.
```

```
Opción: 5
```

```
[MIA] --- Casita.  
Carta superior: 6 de Bastos.  
Tamaño: 7 cartas.  
[RIVAL] --- Casita.  
Carta superior: 9 de Espadas.  
Tamaño: 4 cartas.
```

```
Opción: 3
```

```
Tu mano actual:  
0. 5 de Bastos  
1. 11 de Espadas  
¿Qué carta querés dejar en la mesa?: 1  
Dejaste el 11 de Espadas en la mesa.
```

```
=====
Se terminaron las cartas. Fin de la partida.
Casita Jugador 1: 12 cartas.
Casita Jugador 2: 34 cartas.
=====
----- ¡GANÓ EL JUGADOR 2! -----
```

Conclusión del equipo

La culminación de este proyecto de programación nos permitió validar de forma práctica uno de los pilares fundamentales de la ingeniería de software: un diseño orientado a objetos sólido y bien encapsulado en sus etapas tempranas facilita la extensibilidad del sistema sin necesidad de reescribir por completo el código base. Los componentes desarrollados en la primera versión demostraron su modularidad al acoplarse de buena manera a las mecánicas de la casita robada.

Asimismo, la transición de una lógica centralizada en la mesa hacia un modelo distribuido con controladores independientes (Juego) e interfaces limpias (Menu) nos enseñó la importancia de la cohesión para escribir código mantenible. La manipulación de estructuras de datos dinámicas como `ArrayList` y el análisis de la complejidad temporal al extraer elementos mediante `removeLast()` nos proveyó de herramientas críticas para asegurar la eficiencia del software, demostrando que la abstracción orientada a objetos no sólo imita las interacciones del mundo real, sino que optimiza el rendimiento lógico en la consola.